

Academic Year	Module	Assessment Number	Assessment Type
2025	6CS012 - Artificial Intelligence and Machine Learning.	1	Report

Task 2: Vision Task report

Student Id : 2408616

Student Name : Tenzing kunsang sherpa

Section : L6CG4

Module Leader : Mr.Siman Giri

Tutor : Mr. Anish Khatiwada

Github Link: https://github.com/Tenzingkunsang/AI-Tasks/blob/main/2408616_TenzingkunsangSherpa_visiontask1.ipynb

Abstract

This report looks at image classification on the Flowers Recognition dataset. The dataset has 4,406 pictures of five types of flowers:

1. daisy
2. dandelion
3. rose
4. sunflower
5. tulip

By comparing three methods:

1. A basic CNN model that was trained from the beginning.
2. A complex six-block CNN model that used Batch Normalisation and Dropout. This model was trained with two optimisers.
3. A transfer learning pipeline that used a -trained MobileNetV2 model.

The basic model got 74.36% accuracy on the test data after 716 seconds of training. The complex model did better than the basic model when we adjusted the optimisers. It got 75.26% accuracy with the Adam optimiser and 76.02% with the SGD optimiser. We did a test where we removed Dropout from the complex model. Surprisingly this improved accuracy to 84.31%. This suggests that for this model and dataset Dropout was not helping. The transfer learning method gave the results. A frozen MobileNetV2 model got 88.99% accuracy. When we fine-tuned the 30 layers accuracy went up to 91.40%. In the end this project shows that choosing the right optimiser is important. We also learned that we need to check regularisation strength, for each dataset. Fine-tuning a -trained model takes extra time but it is worth it.

Table of Contents

ABSTRACT	2
1. INTRODUCTION	0
2. DATASET.....	1
2.1 SOURCE AND COMPOSITION	1
2.2 DATA SPLITTING AND PREPROCESSING	2
2.3 DATA AUGMENTATION	3
3. METHODOLOGY.....	4
3.1 BASELINE CNN	4
3.2 DEEPER CNN ARCHITECTURE.....	4
3.3 OPTIMISER COMPARISON: ADAM VS SGD	5
3.4 TRANSFER LEARNING: MOBILENetV2	5
3.5 ROLE OF DROPOUT	6
4. EXPERIMENTS AND RESULTS.....	6
4.1 PERFORMANCE OF THE BASELINE CNN	6
4.2 ADAM VS SGD:	8
4.3 BASELINE VS DEEPER ARCHITECTURE:	9
4.4 ABLATION STUDY: ROLE OF DROPOUT	10
4.5 TRANSFER LEARNING: MOBILENetV2:	11
4.6 MODEL COMPARISON OF THE OVERALL :	14
5. DISCUSSION:.....	15
5.2 R BEHAVIOUR OF THE OPTIMISER AND THE FAIR COMPARISON IMPORTANCE	15
5.3 THE ALTERNATIVE ROLE OF THE DROPOUT	16
5.4: WHY FINE-TUNING MATTERED ON THE TRANSFER LEARNING ?	16
5.5 CONSIDERATIONS OF THE COMPUTATIONS:	17
6. CONCLUSION:	17
6.1 SUMMARY.....	17
6.2 LIMITATIONS.....	17
6.3 FUTURE WORK	ERROR! BOOKMARK NOT DEFINED.

Table of Figures:

FIGURE 1 CLASS DISTRIBUTION ACROSS THE FIVE FLOWER CATRGORIES BAR CHART	2
FIGURE 2 SAMPLE IMAGES OF INE PER CLASS THAT IS IN THE DATASET	3
FIGURE 3 THE DATASET IMAGES UNDER MULTIPLE AUGMENTATION PASSES	3
FIGURE 4 TRAINING AND VALIDATION LOSS AND ACCURACY ACROSS EPOCHS OF THE BASELINE CNN	7
FIGURE 5 CONFUSION MATRIX FOR THE BASELINE CNN	8
FIGURE 6 VALIDATION ACCURACY AND LOSS FOR THE DEEPER MODEL UNDER ADAM AND SGD	9
FIGURE 7 VALIDATION ACCURACY ACROSS THE TRAINING EPOCHS BETWEEN THE BASLINE AND DEEPER ARCHITECTURE.....	10
FIGURE 8 TRANING AND THE VALIDATION CURVES FOR THE DEEPER MODEL WITH THE DROPOUT AND THE WITHOUT IT	11
FIGURE 9 COMBINED TRAINING AND VALIDATION OF THE ACCURACY AND THE LOSS ACROSS BOTH THE MOBILENetV2 PHASE WITH FINE TUNING AND BOUNDARY MARKED.....	12
FIGURE 10 CONFUSION MATRIX OF FOR THE FINE-TUNED MOBILENetV2	13
FIGURE 11 EXAMPLE PREDICTION ON HELD-OUT TEST IMAGES	13

1. Introduction

The identification of a plant species by a photo is at the intersection of computer vision, ecology, and consumer technology. It can be used for biodiversity monitoring, botanical research and applications that aid gardeners and the curious in determining what they are seeing. This was once done with knowledge that most people lacked in. That all changed with the introduction of deep learning and, specifically, CNNs, which enabled the recognition of a high accuracy of classification without a person creating the set of rules for what a 'rose looks like'.

CNNs are useful here because they learn their feature hierarchy from the pixel values themselves, rather than a person having to define the features. Edges and colour gradients are often detected by the early layers followed by shapes, textures and parts of objects in later layers. This makes CNNs appropriate for fine-grained classification problems such as differentiating between flower species, in which the distinctions between the classes are often not very pronounced.

This project is a series of models that are performed on the Flowers Recognition dataset starting from a very basic three block CNN. It explores what happens when it implements depth, the effect of various regularisation choices, the effect of choice of optimiser, and what pre-trained feature extraction really provides. The specific aims are to:

- Design and assess a baseline CNN architecture as a performance bottom-line;
- Discuss how to improve convergence stability and test accuracy by increasing the depth, Batch Normalisation and Dropout; and
- Differs from Adam and SGD at optimising the deeper architecture under a matched training protocol;
- Explore transfer learning with MobileNetV2 (with frozen feature extraction and partial fine-tuning of the backbone);
- **Make practical conclusions about relation between complexity of models, training time, regularisation and final test accuracy.**

2. Dataset

2.1 Source

The data that are used here is the Flowers Recognition data set on Kaggle. It has colour photographs of 5 categories of flowers: daisy, dandelion, rose, sunflower and tulip. These were gathered from real world locations, not from a studio - so there is significant variation in lighting, background noise, scale and angle of view. The images in the dataset directory were scanned and it was found that there are 4406 valid images in the five classes as given in Table 1.

Class	Image Count	Approx. Share
Dandelion	1,109	25.2%
Tulip	1,000	22.7%
Rose	784	17.8%
Daisy	766	17.4%
Sunflower	747	17.0%
Total	4,406	100%

It is a mild class imbalance as dandelion and tulip form almost half the images, while the remaining classes are around 17-18%. This was not fixed by oversampling or class weighting, primarily because none of the classes has fewer than, say, 700 images and a ratio of 1.5 between the largest and smallest class is far from a significant concern for this type of work.

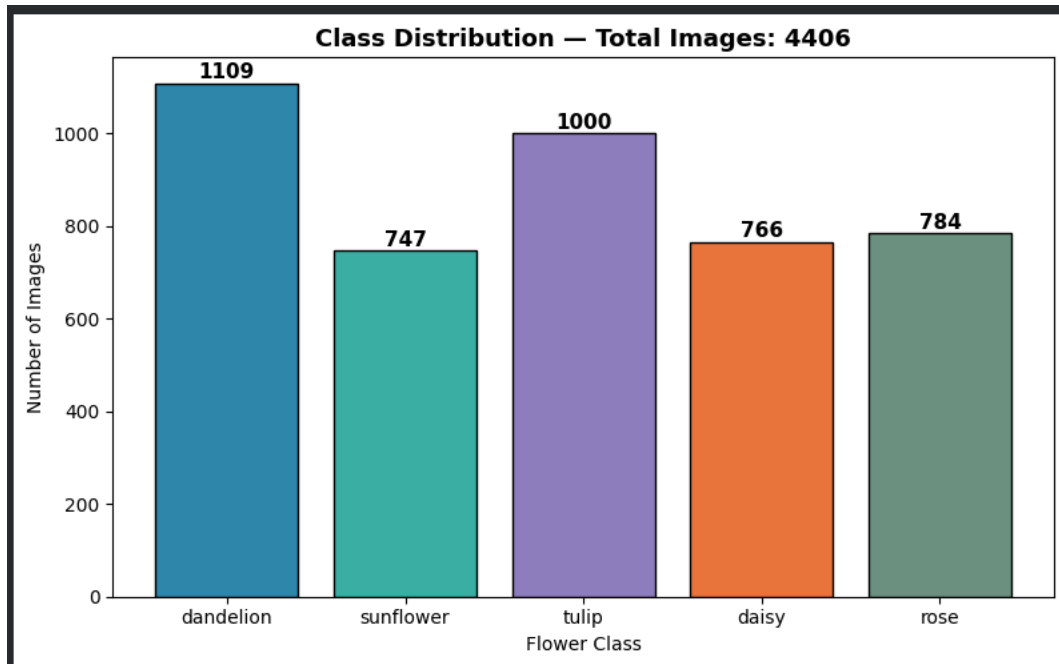


Figure 1 Class distribution across the five flower categories Bar chart

2.2 Data Splitting and Preprocessing

The dataset was split 70/15/15 into training, validation and test sets. Behind the scenes, the images were shuffled prior to splitting so their order on disk did not influence the results, and the split was performed as a one-time copy to new folders with a check to ensure that it wouldn't proceed again during a subsequent run. This left a training set of 3082 images, a validation set of 661 images and a test set of 663 images.

The images of the two CNNs trained from scratch were resized to 128×128 pixels and normalised using a division with 255.0 to range between $[0, 1]$. In the case of MobileNetV2, images were resized to 224×224 to conform to the size of the pre trained backbone, and were also pre processed using its own `preprocess_input` function, rather than simply rescaling, which normalises the values of the pixels to the statistics with which the backbone was trained.

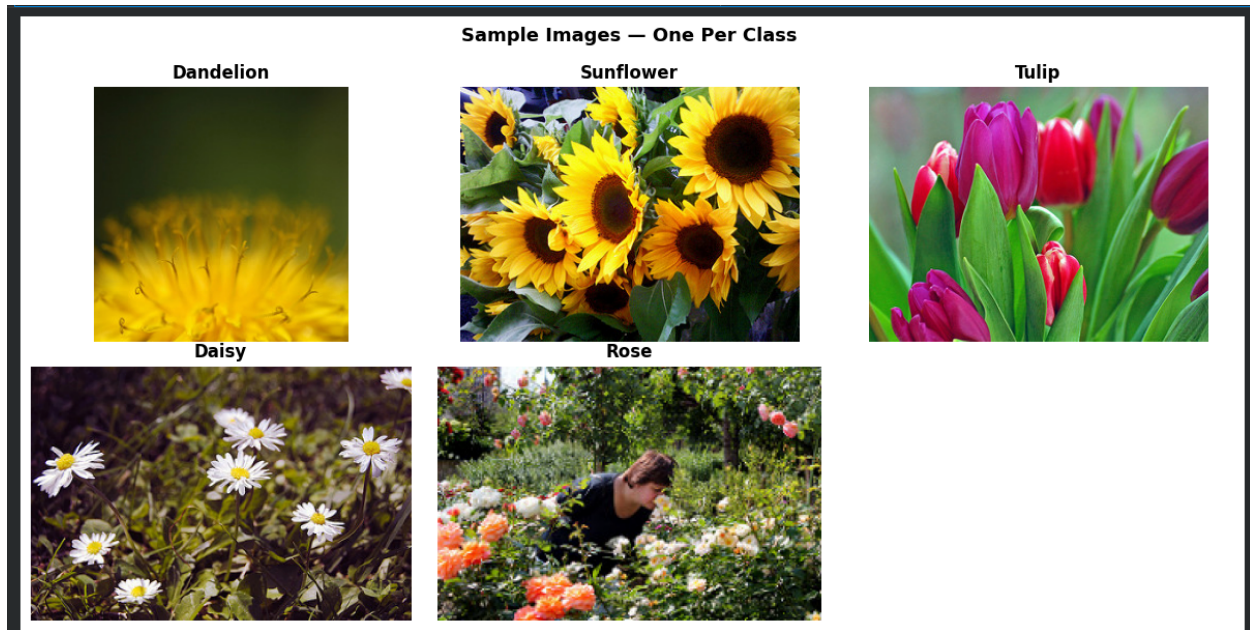


Figure 2 Sample images of ine per class that is in the dataset

2.3 Data Augmentation

Training images were passed through Keras's ImageDataGenerator and were randomly augmented on the fly: rotated up to 30 degrees, sheared up to 20 percent, zoomed up to 20 percent and horizontally shifted up to 20 percent (with nearest neighbour filling for any newly exposed area on the borders). This augmentation has been done on the training generator only. Validation and test generators simply adjusted the images to fit the scale, so the validation and test numbers should just as well be viewed as performance on the real, unaltered photos, and not a "super-augmented" version of them.

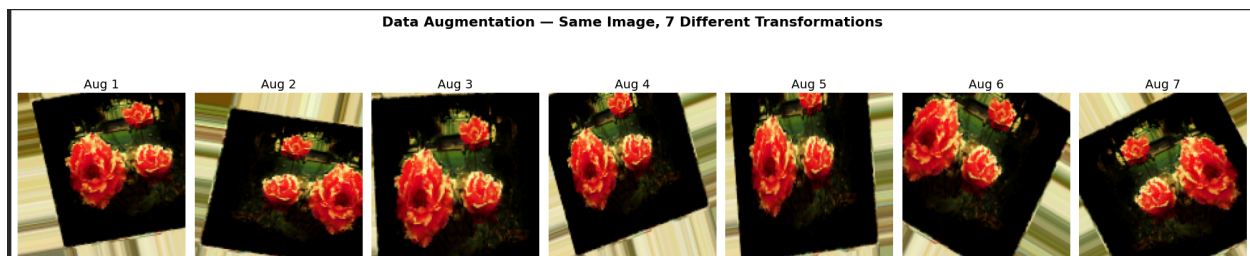


Figure 3 The dataset images under multiple augmentation passes

3. Methodology

3.1 Baseline CNN

The baseline model provides a point by which to compare the more complicated architectures. It is a sequential CNN with three Conv2D layers, followed by three MaxPooling2D layers and ending with a classification head consisting of three Dense layers and a softmax layer that returns the classification. The 3 convolutional blocks are composed of 32, 64 and 128 filters, which have 3×3 kernel, ReLU activation function, and same padding to preserve the spatial dimension (3×3) before pooling. One pooling layer is just a 2×2 window, which decreases the spatial resolution by half. The classification head shrinks through three Dense layers (256, 128 and 64 units, ReLU activation) before the final 5-unit softmax layer. It was compiled with the Adam optimiser and categorical cross-entropy loss and was trained until the validation loss had not improved for 6 epochs, using EarlyStopping with patience=6 and restoring the best weights so as the training would not continue unnecessarily after the validation loss had ceased to improve.

3.2 Deeper CNN Architecture

The deeper architecture is basically a set of six convolutional blocks, growing in the number of filters (32, 64, 128, 256, 512, 512), as specified in the assignment, but without any pooling layer after the last 512-filter block. The concept is that this additional depth enables the network to learn a more rich variety of visual features than can the 3-block baseline model itself.

There were two regularization mechanisms added throughout. For deep networks, the Batch Normalization is located after each convolutional and dense layer, after the linear transformation but before the ReLU activation, to normalize the output of each layer with respect to the results from the previous mini-batch, and to help stabilize the gradients of this layer. Dropout was inserted after each convolutional block (0.2) and after each dense layer in the head following the structure of the baseline, but with a larger number of units (400, 200 and 100) and dropout rate (0.3, 0.3 and 0.3). The whole is packaged as a reusable `build_deeper_model(optimiser)` function, allowing for training of the same architecture with different optimisers for a direct comparison.

In this section, we compare Adam and SGD optimizers . This section compares two optimizers: Adam and SGD.

3.3 Optimiser Comparison: Adam vs SGD

The deeper architecture was trained twice with everything else kept constant (architecture, data generators and EarlyStopping setup), except for the optimizer. To automatically handle individual updates Adam used a learning rate of 0.0005 which is based on the adaptive per-parameter scaling. For SGD, a very large learning rate (0.02) and momentum (0.9) had to be used. That is not a mistake, that's on purpose. The updates were too small to make any progress in a similar number of epochs, as SGD did not have any built-in mechanism to adapt the step size to the shape of the loss surface, so it was tested initially with Adam's learning rate (0.0005). It was actually at 0.02 momentum that it was able to walk through the loss landscape adequately. Both runs had the ReduceLROnPlateau decay learning rate when validation loss did not improve, but SGD has a gentler decay schedule as compared to Adam (factor 0.7, patience 6, floor 1×10^{-4} for SGD vs. factor 0.5, patience 3, floor 1×10^{-3} for Adam). SGD's calcars would take longer to settle and are more prone to noise, so if it was the wrong learning rate, it could find itself in a negative learning scenario.

3.4 Transfer Learning: MobileNetV2

Transfer learning is a process that is used to transfer feature representations learned from a big general-purpose dataset to a small, specific task dataset, rather than learning everything from a random starting point. Here, MobileNetV2, a well-trained model on the 1.2-million-image, 1,000-class ImageNet dataset, was selected as the backbone. Its depth-wise separable convolutions and inverted residual blocks with linear bottlenecks have an excellent accuracy-to-parameter ratio compared to larger architectures such as VGG16 or ResNet50, and are easy to fine-tune under the time and hardware constraints of this assignment.

This has been constructed in two stages. The convolutional base was pre-trained using the ImageNet classification task and was loaded in the feature-extraction phase with the pre-trained ImageNet weights and without the classification head (`include_top=False`). The entire base was

frozen, followed by a new head consisting of a 200-unit Dense layer with ReLU, a Dropout layer (0.3) and a final SoftMax layer of 5 units. This new head was the only one that was trained with learning rate 0.0005 for up to 20 epochs, and the training was stopped by the EarlyStopping (patience 6). The final 30 layers of the 154-layer base model (MobileNetV2) were unfrozen during the fine-tuning phase, and the learning rate during the retraining was set to 1×10^{-5} , which is a much smaller amount than during base model training (around 2.5 million parameters, with trainable parameters of about 257,000). This recompilation step is needed whenever changes are made in the trainable layers in Kera's, and the low learning rate is important because it ensures that the pre-trained weights are not completely erased when training, but rather nudged just a bit. This second phase lasted another 15 epochs using its own EarlyStopping callback (patience 5).

3.5 Role of Dropout

An removal study was performed to determine what Dropout was doing in the deeper architecture, by removing all Dropout layers while leaving Batch Normalisation, architecture depth and all training conditions the same as the Adam-optimised deeper architecture. This way, if there is a difference in result, it is easier to attribute the problem to Dropout and not other factors such as using a different optimiser, a different learning rate schedule or a different number of epochs.

4. Experiments and Results

4.1 Performance of the Baseline CNN

The CNN model was trained for a maximum of 30 epochs and an EarlyStopping function was used to stop training if the validation loss did not increase. It took 716.24 seconds to train and had a test loss of 0.6918 and test accuracy of 74.36%. That is a good lower bound: a simple CNN with no regularization other than early stopping achieves approximately 75% correct rate for test images.

The per-class report on the table 2, Shows performance isn't even across categories. Daisy has high precision (0.86) and low recall (0.77) – the model is cautious when labeling something a daisy and it tends to label daisies as something else more often than not. However, rose is the

poorest class (F1-score 0.64) of all, which is consistent with the fact that roses can look very similar to other, flower-based classes, under certain angles and lighting conditions. Sunflower is the most easily recognized (0.85); likely due to its color, yellow petals and dark center are difficult to mistake for any other plant in this dataset.

Class	Precision	Recall	F1-score
Daisy	0.86	0.77	0.82
Dandelion	0.80	0.82	0.81
Rose	0.64	0.64	0.64
Sunflower	0.70	0.85	0.76
Tulip	0.73	0.63	0.68

Table 2. Per-class precision, recall and F1-score for the baseline CNN on the test set.

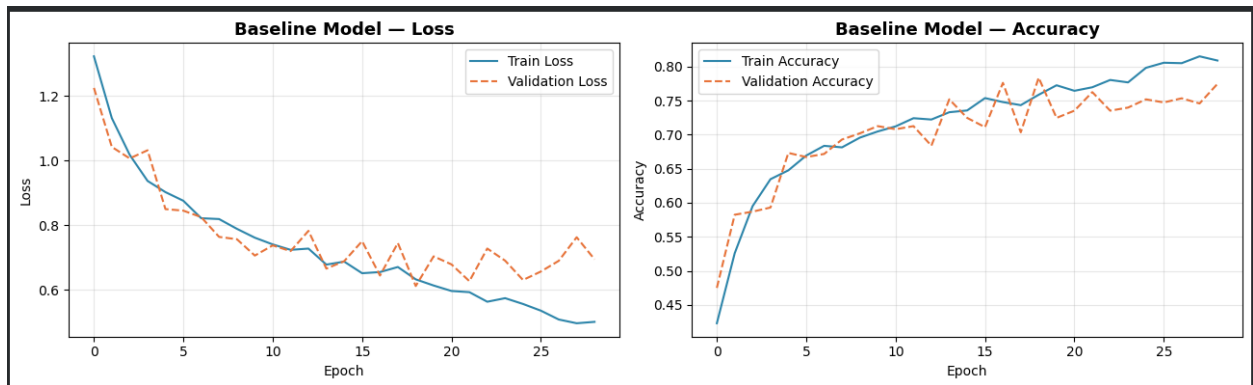


Figure 4 Training and validation loss and accuracy across epochs of the Baseline CNN

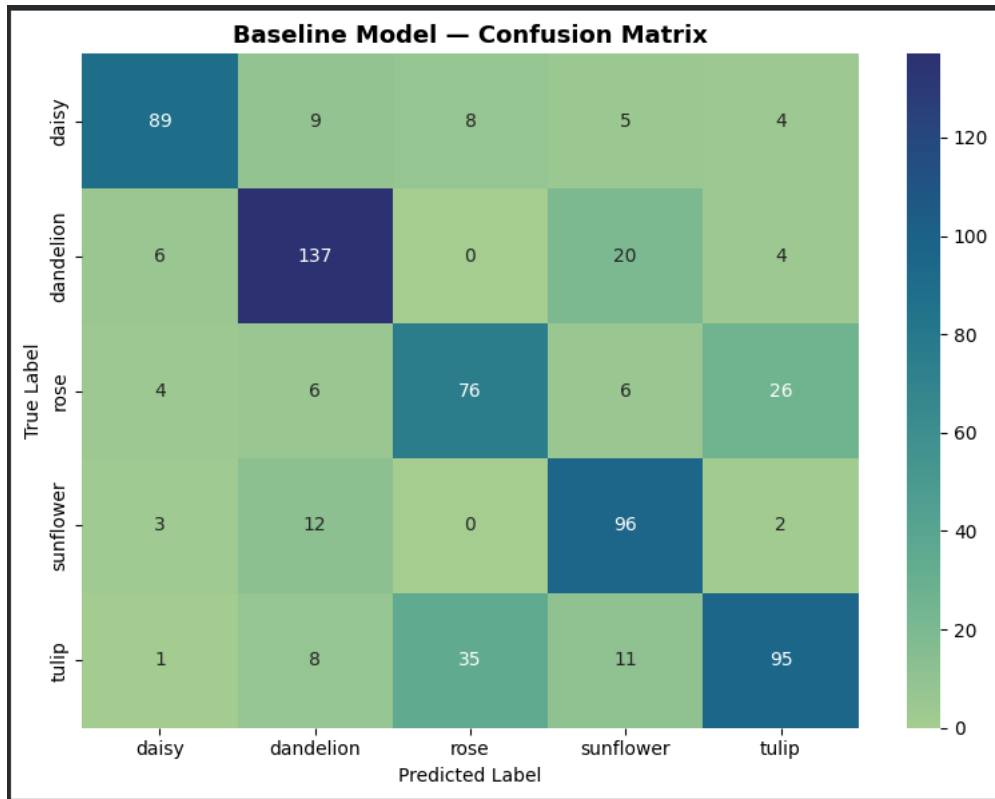


Figure 5 Confusion Matrix for the baseline CNN

4.2 Adam vs SGD:

After resolving the learning rate mismatch issue from Section 3.3, the learning rate of both optimizers stabilized and were trained in a similar manner, not in one way being the sole winner. Adam reached 75.26% test accuracy (loss 0.6914) in 905.63 seconds; SGD reached a slightly higher 76.02% (loss 0.6723) in a similar 894.73 seconds. This is a completely different story from an earlier training session where SGD has the same learning rate as Adam. In that setup, SGD's validation loss fluctuated and didn't show any clear trend downwards, and EarlyStopping ended training early before there was any real progress. Such a previous result didn't mean that SGD is a poor optimizer, but it did mean the learning rate was not appropriate for SGD. This is more than just this experiment; the choice of learning rate can give the wrong answer when comparing optimizers at the same hyperparameter values: the SGD with no adaptive scaling would require an extremely different learning rate to even be in the same ballpark as Adam. Configure it with a more reasonable rate (0.02 vs 0.0005) and a less aggressive decay schedule and it gets about as accurate as Adam could have been, plus it gets a little bit lower test loss.

This matters beyond just this one experiment: comparing optimisers at identical hyperparameter settings can give a misleading answer, since SGD without any adaptive scaling needs a very different learning rate to even be in the same ballpark as Adam. Give it a properly scaled rate (0.02 instead of 0.0005) and a gentler decay schedule, and it matches Adam's final accuracy almost exactly, with a slightly lower test loss to boot.

Optimiser	Test Accuracy	Test Loss	Training Time (s)
Adam	75.26%	0.6914	905.63
SGD	76.02%	0.6723	894.73

Table 3. Optimiser comparison for the deeper CNN architecture.

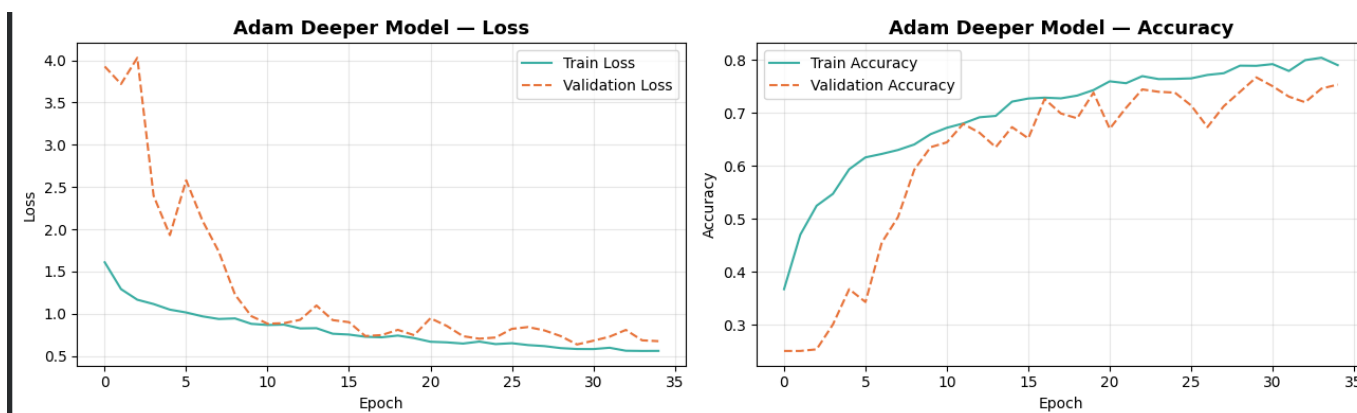


Figure 6 Validation accuracy and loss for the deeper model under Adam and SGD

4.3 Baseline vs Deeper Architecture:

Training the CNN properly, the model could outperform the baseline, as can be seen in the comparison between the simplest and most complex from-scratch CNN, with a score of 75.26% (Adam) and 76.02% (SGD) respectively, against the baseline of 74.36%. One to two percentage points is a very narrow range and should not be interpreted as evidence of large gains due to depth alone on this data set. This is only about 3082 images (five classes) so it is not very many for a six block network with up to 512 filters per layer. Batch Normalisation and Dropout were required to make this additional capacity usable at all, and even then the added value compared to the base model is not that spectacular. This is the accepted wisdom: the more data there is, the

more capacity is needed in the model or they will not be worth much until there are more data or there is more tuning.

Model	Test Accuracy	Test Loss	Training Time (s)
Baseline CNN	74.36%	0.6918	716.24
Deeper CNN (Adam)	75.26%	0.6914	905.63
Deeper CNN (SGD)	76.02%	0.6723	894.73

Table 4. Comparison of baseline and deeper architectures across both optimisers.

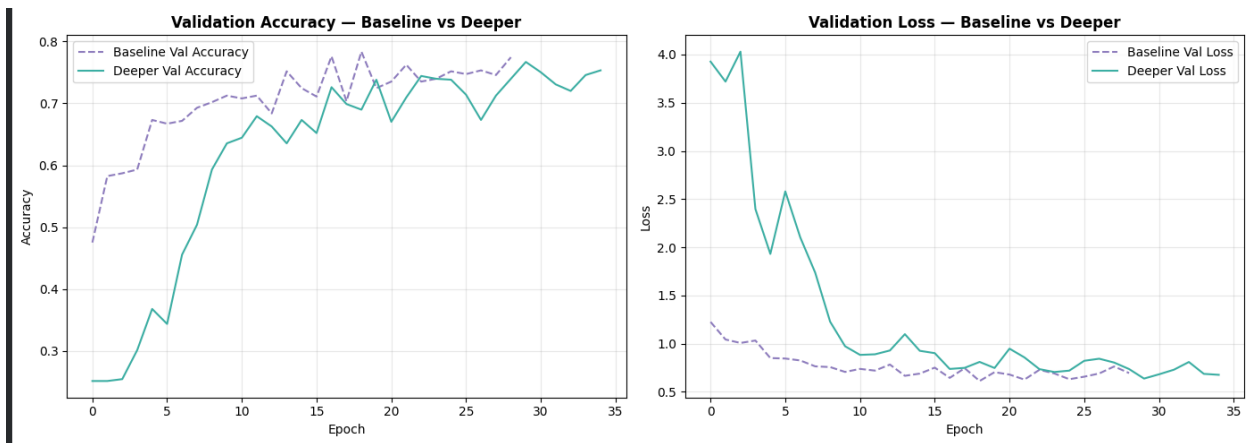


Figure 7 Validation accuracy across the training epochs between the baseline and deeper architecture

4.4 Ablation Study: Role of Dropout

Keeping the Batch Normalization layers and the deeper architecture the same, but removing all the Dropout layers, increased test accuracy to 84.31% (loss 0.4533) which represents approximately 9 points higher than the version that included Dropout layers. This may seem like going backwards, as Dropout is usually added to prevent overfitting and enhance generalization. The two factors probably account for it. One thing is that the 0.2 drop rate after each of the 6 convolutional blocks, with a further drop rate of 0.3 after each of the dense layers in the head, is quite a significant regularization burden. When stacked that many layers, it's possible that useful capacity is lost overall, and not just prevented from overfitting, so some of the benefit of adding extra layers in Section 3.2 is just wasted. Second, Batch Normalization is good on its own, because it adds noise to the training using batch level statistics. If both mechanisms are used together, Dropout's additional contribution might be less than if it were used by itself, but it uses the same capacity.

But it does demonstrate that the strength of the regularization is a hyperparameter, and it is one that needs to be checked, it does not automatically need to be cranked up. As a next step it might be interesting to explore a variety of Dropout rates, and to determine if a lower Dropout rate (e.g. 0.1 vs. 0.2-0.3) achieves similar regularization properties with less capacity-sacrifice.

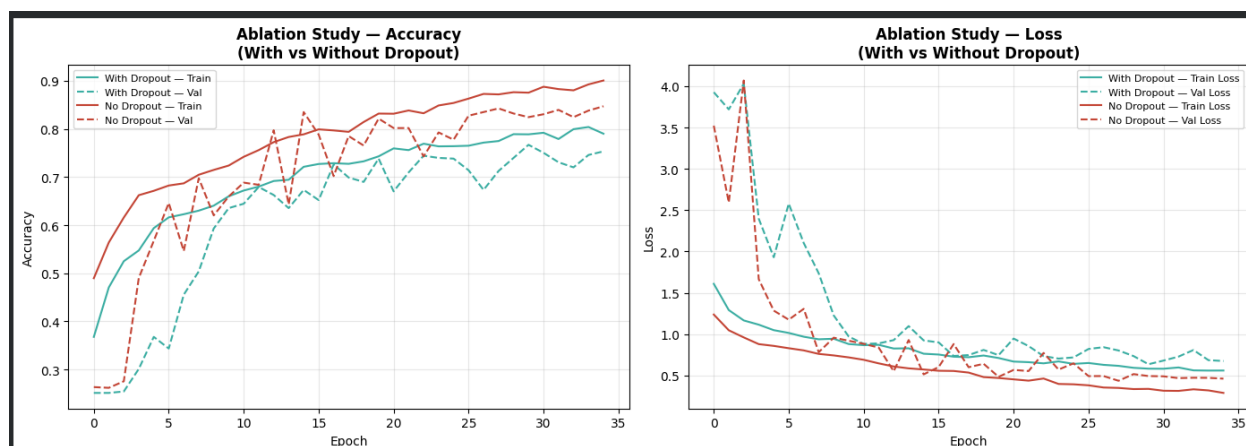


Figure 8 Training and the validation curves for the deeper model with the dropout and the without it

4.5 Transfer Learning: MobileNetV2:

The MobileNetV2's base model was able to tackle the feature-extraction stage of the model, achieving a test accuracy of 88.99% (test loss of 0.2918) after 742.22 seconds, significantly better than all of the models that were trained from scratch in this project, even the best of the deeper architecture. No surprise, the payoff of features trained on 1.2 million ImageNet images: edges, textures and shape primitives are reasonably transferable to a new domain, one which was not explicitly represented in the pre-training domain (flower photos).

Unfreezing the last 30 layers for base MobileNetV2 and training with 1×10^{-5} learning rate resulted in an even higher test accuracy of 91.40% (loss 0.2649), but at a much higher price in terms of time – 1,546.35 seconds across both unfreeze and training phases. The training curves for this phase exhibit a slight drop in the accuracy of the validation immediately after the

unfreezing, as the previously frozen layers begin to start adjusting to performing gradient updates for the first time, followed by a subsequent rise in accuracy, and ultimately surpassing the feature-extraction-only result within a few epochs. It is possible that fine-tuning is working exactly as it should, that is, a small instability that is low in learning rate and followed by a recovery to a more satisfactory result, rather than its being the result of something gone wrong, which would be indicated by a larger, unrecoverable instability.

Stage	Test Accuracy	Test Loss	Training Time (s)
Phase 1 (Feature Extraction)	88.99%	0.2918	742.22
Phase 2 (Fine-Tuned)	91.40%	0.2649	1,546.35 (cumulative)

Table 5. MobileNetV2 results, feature extraction versus fine-tuning.

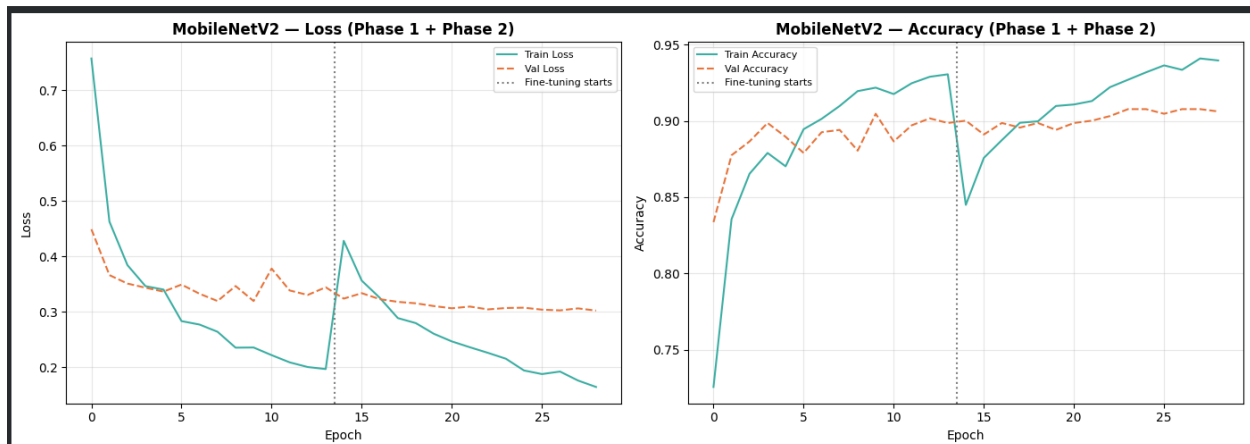


Figure 9 Combined training and validation of the accuracy and the loss across both the MobileNetV2 phase with fine tuning and boundary marked

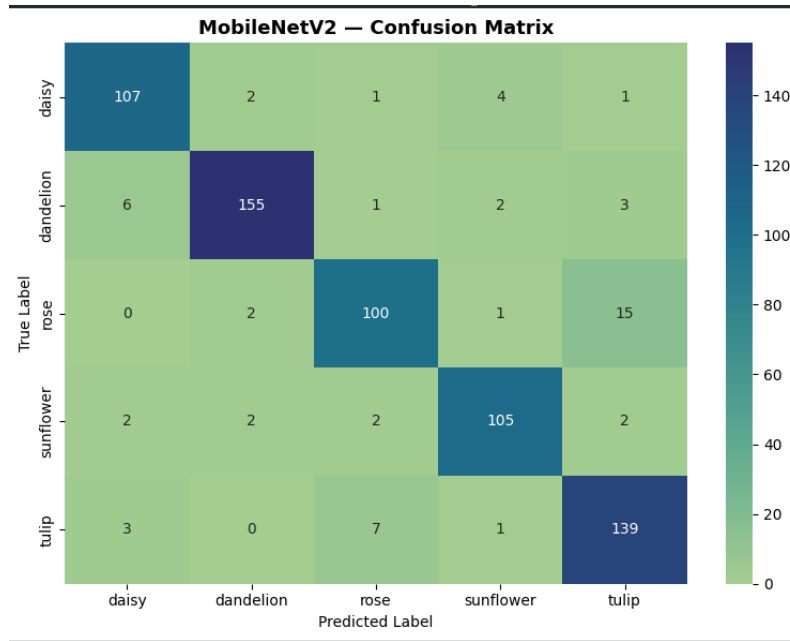


Figure 10 Confusion matrix of for the fine-tuned MobileNetV2

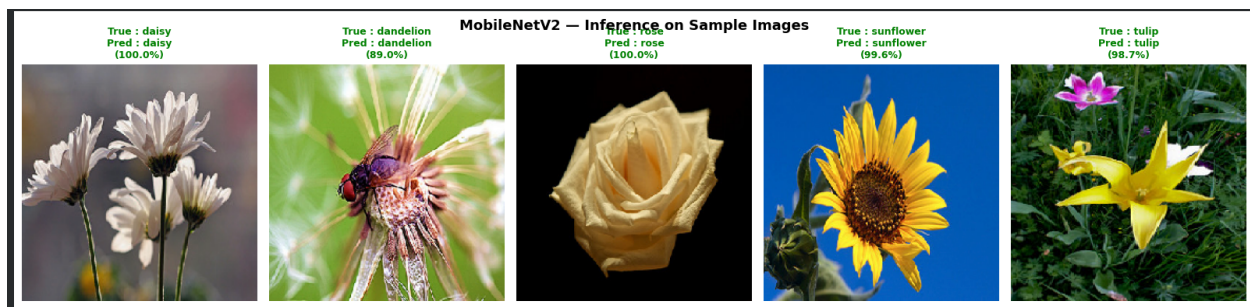


Figure 11 Example prediction on held-out test images

4.6 Model Comparison of the Overall :

All of the configurations attempted for this study are listed in Table 6. The most prominent finding is that transfer learning, and fine-tuning, in particular, provides the largest individual gain in this project. That agrees with the typical transfer learning literature on the small image classification problems from a narrow domain where a few thousand is not usually sufficient to train similarly strong features from random initializations.

Model	Test Accuracy	Test Loss	Train Time (s)
Baseline CNN (scratch)	74.36%	0.6918	716.24
Deeper CNN + Adam	75.26%	0.6914	905.63
Deeper CNN + SGD	76.02%	0.6723	894.73
Deeper CNN, no Dropout (ablation)	84.31%	0.4533	n/a*
MobileNetV2 — Feature Extraction	88.99%	0.2918	742.22
MobileNetV2 — Fine-Tuned	91.40%	0.2649	1,546.35

Table 6. Complete comparison across all six trained configurations.

5. Discussion:

The thing for the most question of the part was, “Does having more depth always work?”. This isn't a setup that is losing to its base model straight up, it is a genuine improvement, albeit minor, once the optimiser was set up correctly. Interesting: it's not validated or invalidated by this data, but it's not even proven wrong per se that ‘deeper is always better’. Despite adding two different types of regularisation and increasing the number of convolutional blocks from three to six, there's not been a significant amount of change in terms of the percentage of accuracy. With that, even with the two separate regularisation techniques and the number of convolutional blocks increasing from three to six, there is not much of a difference in the percentage of accuracy. The approximate 3,000 images for each of five classes appear to set a hard limit on how much extra capacity is worth having. This is consistent with the general principle that the capacity of the model should grow with the size of the data set, and the deeper architecture would likely be able to perform a lot more than the baseline does here, if it were trained on a larger or more diverse data set.

5.2 Behaviour of the Optimiser and the Fair Comparison importance

I think the main takeout from this project is rather how you compare optimizers in the first place. It was very deceiving to try Adam and SGD with the same learning rate, and SGD appeared to have failed completely. It wasn't because SGD was not a good choice for this task – it was because a learning rate that was optimized for Adam's adaptive update rule is too small to make a comparable amount of progress in the same number of epochs while using SGD's fixed-rate update rule. With an increasingly noisy convergence, SGD obtained a much higher learning rate (0.02 compared to Adam's 0.0005), and nearly caught up with Adam on final test accuracy (76.02% vs. 75.26%).

This isn't a lesson just for this particular dataset; if the optimizer comparison is to have meaning, each optimizer needs to be given a fair chance of converging, which is achieved in practice by adjusting their hyperparameters separately from each other instead of using one set of hyperparameters across all optimizers that update differently. I'm not sure that it's really comparing two optimizers when the learning rate is the same.

5.3 The alternative role of the Dropout

The most interesting finding in this whole project is that the ablation result – removing Dropout improving test accuracy – is exactly the opposite of what one would expect from the textbook that Dropout should reliably improve generalization. The most probable reason is that here the Dropout on top of the Batch Normalization is applied on 6 convolutional blocks and 3 dense layers, and at the rates set (0.2 for Dropout in the convolutional blocks and 0.3 for Dense layers), the regularization pressure becomes greater on the useful capacity than the harmful overfitting. It's a helpful warning that the regularization parameters you use are not fixed parameters that you can apply to any architecture, other regularization, or any dataset size; you have to test a specific combination of them to see how they affect the results.

5.4: Why Fine-Tuning Mattered on the Transfer learning ?.

The transfer learning method outperformed the other methods in this research by large margin, and the difference between feature extraction (88.99%) and the partial fine-tuning (91.40%) indicates the importance of doing the partial fine tuning in contrast to feature extraction when sufficient data and time are available. When an image backbone is frozen, only a small head on top of generic ImageNet features can be learned. While the features are transferable to flower images, the level of transferability is roughly that of natural images, that is, textures, colors and shape boundaries learned from natural images generalize fairly broadly; however, the features were not specifically tuned to differentiate flower species from one another. The network's higher-level features were trained to specifically look at flowers by training the network on only the last 30 layers, and finally the very low fine-tuning learning rate (1×10^{-5}) prevented the specialization from eradicating the useful pre-trained layers. A quick drop in validation accuracy immediately following unfreezing, then rising to a higher level is expected, not a sign of a problem.

5.5 Considerations of the computations:

All the trainings were performed on Google Colab with GPU acceleration. Training time was more correlated with the model complexity and input resolution than it was with the number of parameters in the models: the smaller CNN was fastest at 716 seconds; the deeper one took about 900 seconds using either optimiser; the two stages of the MobileNetV2 were roughly 1,546 seconds. The latter number is mostly due to the larger input resolution of the pre-trained backbone (224×224) rather than on the number of the trainable parameters, which was considerably lower than either of the pre-trained models in the feature-extraction phase. While these training times work in a coursework setting, there is a real cost here – the best-performing setup (MobileNetV2 fine-tuned) seemed to cost over double that of the baseline.

6. Conclusion:

6.1 Summary

Utilizing the Flowers Recognition dataset, a project evaluated five flower types by comparing three models: essentially an uncomplicated CNN model with and without Dropout testing using two distinct methods, followed by transfer learning-based models using the MobileNetV2 model. The basic CNN achieved 74.99% accuracy on the test, while the deeper model was adjusted properly with Adam and SGD, leading to better accuracy.

6.2 Limitations

Due to the limited number of training images available for this dataset across five classes, the problem may be minimal in terms of deep learning standards. Additionally, some of the most intriguing findings from the report, such as the small gap between baseline and deeper architectures and negative interaction between Dropout and Batch Normalisation, may not be feasible on a larger dataset, where additional model capacity and stronger regularisation might pay off more easily.